

Rendu ISA Devops Skimaster - Slides Augmentés

Membre de l'équipe :

- Driss Meskini (MkDriss)
- Cyril Rouillon (Mus)
- Dorian Reynier (Fenris1801)
- Léo Migny (MGS)
- Ludovic Clolot (Ludoclt)
- Aurélien Gorjux (¯_(\ツ)_/_)



ISA - Sommaire

- **Architecture**
- **Diagramme de composant**
- **Interfaces**
- **Diagramme de classe**
- **Persistance**
- **Scénarios**

ISA - Architecture

Forces :

- Bonne décomposition des responsabilités, ce qui nous a permis d'ajouter des features simplement
- Gestion des gates, facile à ajouter et contraintes réseaux prise en compte

Capacité d'évolution :

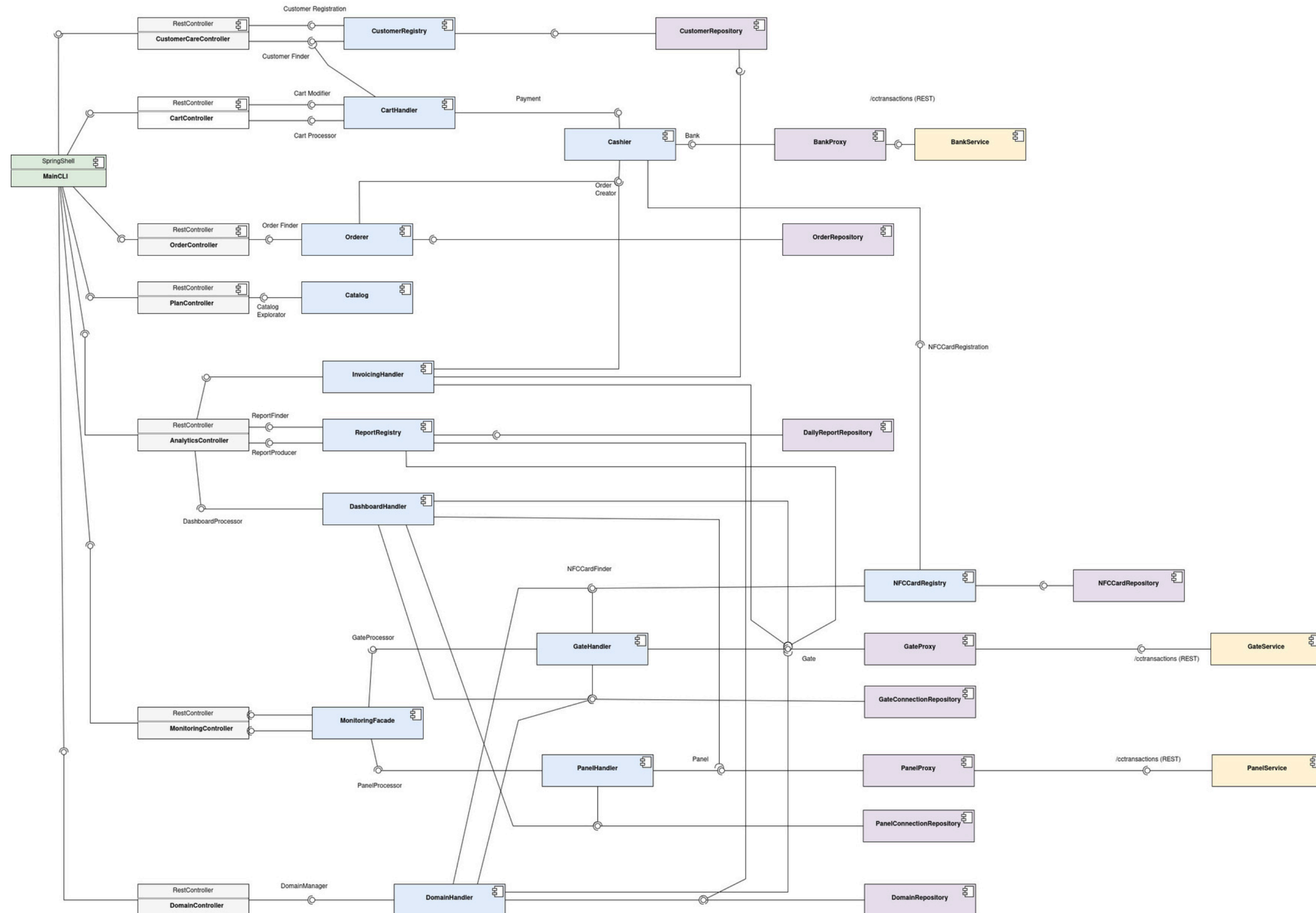
- Développer la capacité de test

Faiblesses :

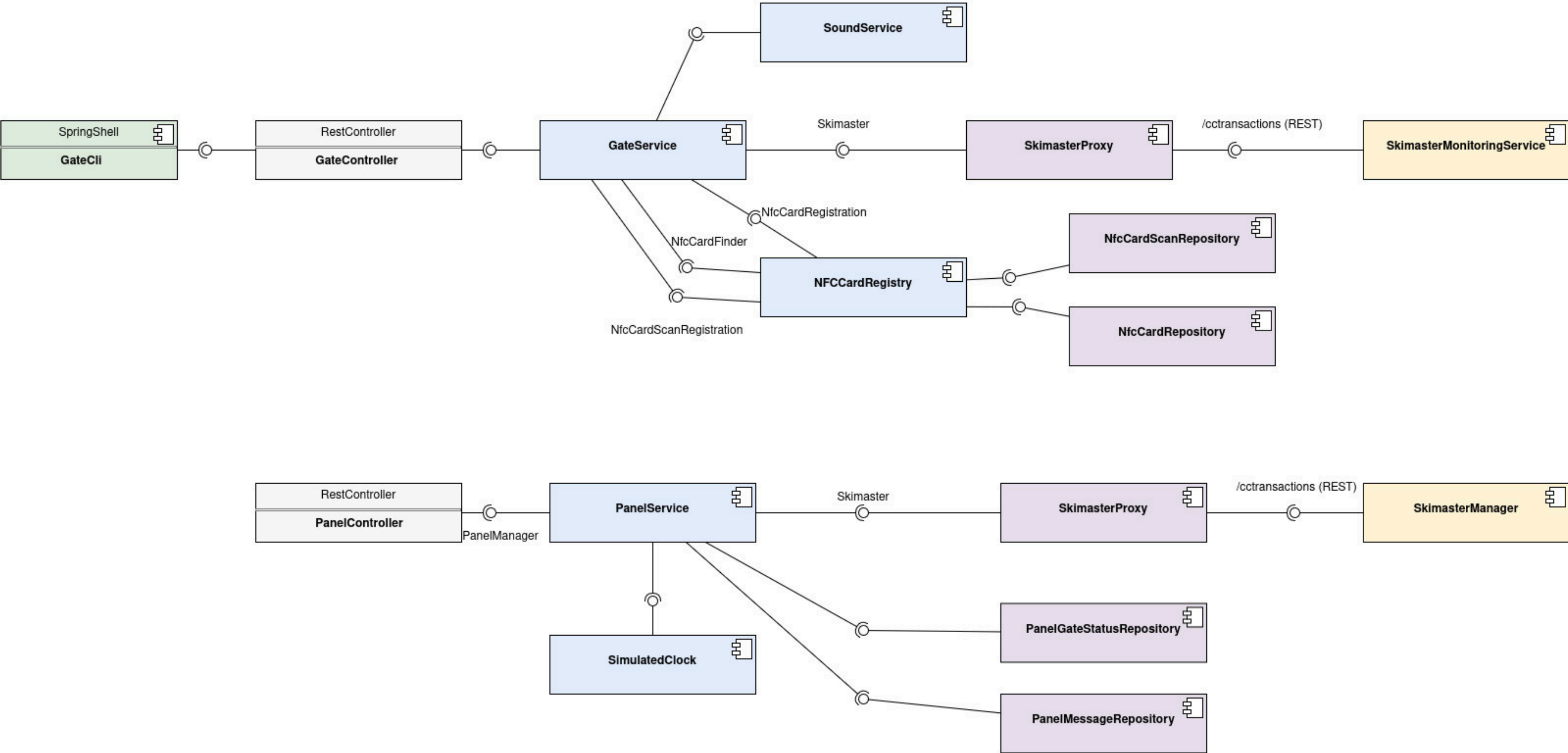
- Test d'intégration qui pourraient être plus complet
- Dashboard pourrait être vu comme un système externe et pas directement dans le backend

ISA - Diagramme de composants (vue d'ensemble Skimaster)

Diagramme complet: [https://github.com/pns-isa-devops/isa-devops-25-26-isa-25-26-team-i/blob/fix/clean-code-for-demo/docs/component diagram.jpg](https://github.com/pns-isa-devops/isa-devops-25-26-isa-25-26-team-i/blob/fix/clean-code-for-demo/docs/component%20diagram.jpg)



ISA - Diagramme de composants (vue d'ensemble Gate & Panel)



ISA - Diagramme de composant

Afin de découper au mieux les responsabilités métiers de notre application, nous avons utilisé 4 systèmes qui interagissent entre eux. Chaque portique et chaque panel de la station sont des machines indépendantes embarquant leur propre service.

- Backend : contenant la logique principale de la gestion de Skimaster
- Bank : Banque issue de TCFS
- Gate : Une porte de la station
- Panel : Un panel de la station

Nous avons également ajouté des CLIs permettant de communiquer avec nos systèmes :

- CLI : Communique directement avec le backend en appelant diverses controllers
- Gate-CLI : Communique directement avec la gate qui lui est associée (voir docker-compose)

Idée générale de l'architecture :

Pour qu'un système externe puisse communiquer avec notre application, nous avons mis en place des @RestController via SpringBoot, permettant d'exposer des endpoints via une API REST. Ces controllers reçoivent les requêtes externes et délèguent leur traitement aux @Service ou @Component appropriés, puis renvoient une ResponseEntity contenant le résultat de l'opération. Ce sont ces mêmes services qui sont liés aux @Repository permettant la persistance des données. Nous reviendrons sur la persistance un peu plus tard.

ISA - Diagramme de composant

Pour communiquer avec les systèmes externes, notre Backend utilise des interfaces appelées Proxy (BankProxy, GateProxy, PanelProxy) représentant respectivement la banque, un portique ou un panneau. C'est au travers de ces interfaces que nous effectuons les appels REST vers les systèmes visés. Cela permet d'abstraire les détails de communication réseau derrière une interface claire, et faciliter le remplacement ou le mock de ces systèmes lors des tests. Symétriquement, Gate et Panel disposent chacun d'un SkimasterProxy pour contacter le Backend.

Les informations sont communiquées via des DTO, exposant uniquement les données que l'on souhaite transmettre et non la classe entière.

Cette architecture permet de séparer clairement les responsabilités entre les différents systèmes, de faciliter la maintenabilité du code et de simuler des systèmes externes indépendants communiquant via une API REST.

Backend

Le backend contient la majorité des composants métiers de l'application, organisés selon une séparation entre exposition de l'API, logique métier et persistance des données.

Les RestControllers exposent les différentes fonctionnalités via l'API REST et délèguent leur traitement aux composants métiers. Plutôt que de coupler directement les controllers à l'ensemble des handlers, certains composants jouent un rôle de façade (comme la MonitoringFacade) offrant un point d'entrée unique et réduisant le couplage entre l'exposition de l'API et l'orchestration interne.

ISA - Diagramme de composant

Les handlers et registries (par exemple GateHandler, Orderer, CustomerRegistry ou NFCCardRegistry) implémentent la logique métier principale. Les handlers coordonnent plusieurs sous-composants pour réaliser des opérations complexes, tandis que les registries centralisent la gestion d'un type d'entité précis, permettant une interface cohérente pour la recherche et l'enregistrement. Ces composants s'appuient ensuite sur des @Repository pour assurer la persistance des données.

Gate

Le système Gate représente un portique d'une remonté mécanique. Il gère les interactions liées au passage des utilisateurs et du perchiste. Il contient notamment un GateService chargé de la gestion principale du portique. Il stocke en local les cartes nfc qui sont acceptées sur ce portique. Le Backend peut par ailleurs, venir mettre à jour les cartes disponibles en fonction du domaine auquel la gate est rattachée.

Panel

Le système Panel représente un panneau d'information de la station. Il permet d'afficher différentes informations aux utilisateurs via un PanelService, et utilise des composants utilitaires comme une SimulatedClock pour gérer certains événements temporels.

ISA - Interfaces metier (Extraites de TSCF)

Interfaces de Cashier :

- Payment : operations related to the payment of a given cart's contents

Interfaces de Customer :

- CartModifier : operation to modify a given customer's cart, by updating the number of ski passes in it, and to retrieve the contents of the cart
- CartProcessor : operation for computing the cart's price and validating it to process the associated order
- CatalogExplorator :
- CustomerFinder :
- CustomerRegistration :

Interfaces de Order :

- OrderCreator :
- OrderFinder: a finder interface to retrieve an order based on its identifier, to follow its status

ISA - Interfaces metier (Backend)

Interfaces de Monitoring (1):

- Dashboard Processor : a processor interface to manage domain operations, including registering gates, setting up card distributions to gates, adding cards to gates, adding plans to domains, listing and creating domains
- Domain Manager : a manager interface to manage domain operations, including creating domains, listing all domains, assigning ski plans to domains, setting up and distributing NFC cards across domains, registering gates to domains, and adding NFC cards to all gates within domains
- GateProcessor : a processor interface to manage gate operations, including configuring thresholds, retrieving gate status, opening or closing gates manually
- PanelProcessor : a processor interface to manage panel operations, including reading messages, writing to panels, updating gate statuses, and adding/removing gate statuses from the display

ISA - Interfaces metier (Backend)

Interfaces de Monitoring (2):

- ReportFinder : a finder interface to retrieve resort daily reports by date and manage their lifecycle, allowing to find a report by day, delete today's report, and save reports
- ReportProducer : a producer interface to generate resort and domain daily reports from gate connections, and convert report entities to their DTO representations for API responses
- Gate : a connector interface to communicate with physical gate microservices, allowing to send commands (open/close), set thresholds, refresh card lists, and request status/reports
- Panel: a connector interface to communicate with physical panel microservices, allowing to read messages, write messages, update and manage gate statuses on panels

ISA - Interfaces metier (Backend)

Interfaces de Nfc :

- NFCCardRegistration: a registration interface to register new NFC cards for customers with a specific plan and sound profile
- NFCCardFinder : a finder interface to retrieve NFC cards by various criteria (plan, customer, ID), enabling card lookups across the system

ISA - Interfaces metier (Gate & Panel)

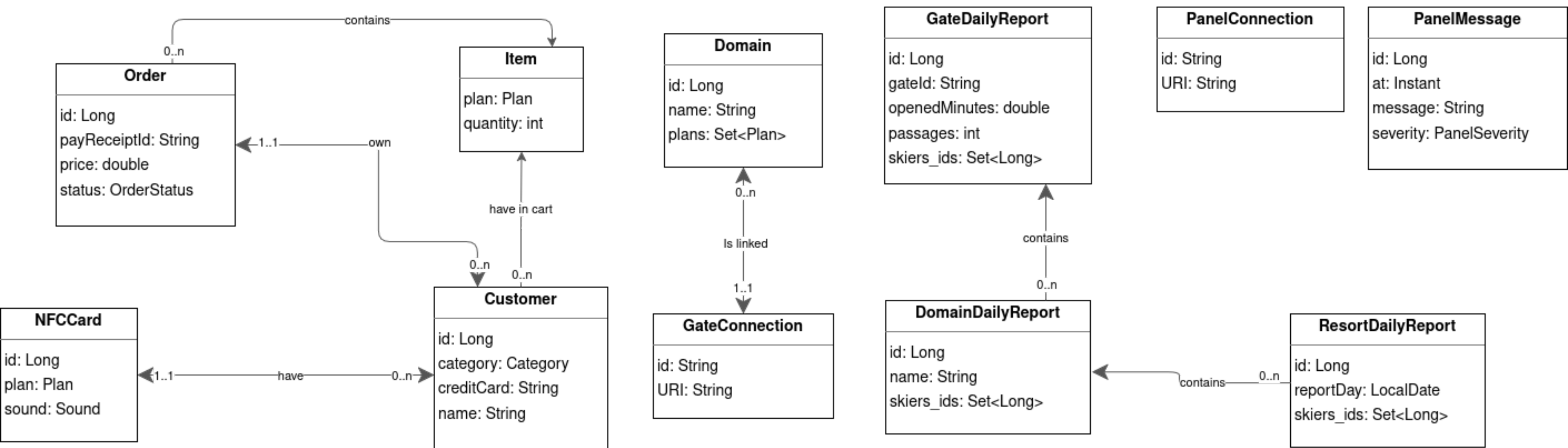
Interfaces de Gate :

- NfcCardFinder : a finder interface to retrieve NFC cards stored locally at the gate, including finding cards by plan and retrieving today's SUPER_CARD scans
- NFCCardRegistration : Similaire au NFCCardRegistration de monitoring dnas le backend mais local pour la gate
- NfcCardScanRegistration : a registration interface to record NFC card scans at gates with timestamp and threshold checks
- Skimaster : a connector interface used by gate microservices to notify the backend about gate events (registration, issues, status changes, threshold alerts)

Interfaces de Panel :

- PanelManager : panel microservices to register with the backend and fetch current gate statuses for display

ISA - Diagramme de classe du modèle métier



ISA - Persistence

Backend

Côté backend, nous avons conservé la base fournie par la TCFS en persistant les Customer et les Order validées. À l'issue d'un achat de forfait, les NFCCards sont également persistées afin de pouvoir être transférées vers les gates à chaque démarrage ou redémarrage de la station.

Nous persistons également les GateConnections et PanelConnections afin de ne pas avoir à reconfigurer les gates et panels à chaque démarrage de l'application. Enfin, les DailyReport sont stockés de manière persistante pour permettre leur consultation ultérieurement.

Gate

Côté gate, les cartes NFC autorisées à passer sur la gate concernée sont persistées localement. Par défaut, la gate est fermée et doit être ouverte manuellement, soit depuis la gate elle-même, soit depuis la station de contrôle. Ce choix reflète notre volonté d'avoir un comportement explicite et sécurisé lors d'un redémarrage, plutôt que d'ouvrir automatiquement les portiques.

Les passages sur la gate sont également persistés localement, afin de pouvoir restituer la liste complète des passages en fin de journée même en cas de défaillance de la gate en cours de journée.

Panel

Côté panel, nous persistons les liaisons aux gates via les GateStatus, de façon à n'avoir à configurer qu'une seule fois les gates associées au panel. Les messages envoyés depuis la station de contrôle sont également stockés, permettant de conserver un historique complet des informations remontées sur le panneau.

ISA - Scénarios

Scénario 1 :

- Setup de la station et des domaines dont 1 domaine “Beginner Pass” & achat de plans dont 1 “Beginner Pass”

Scénario 2 :

- Scan de cartes nfcs sur différents domaines, dont 1 “Beginner Pass”

Scénario 3 :

- Achat d’un pack famille
- Met à jour les gates (pendant que la station est ouverte)
- Vérification de la sonorité “Low” pour un enfant et “High” pour un adulte

Scénario 4 :

- Facturation des supercartes du jour

Scénario 5:

- Perchiste remonte un évènement et ferme la gate
- Re Ouverture de la gate depuis la station de contrôle

Scénario 6:

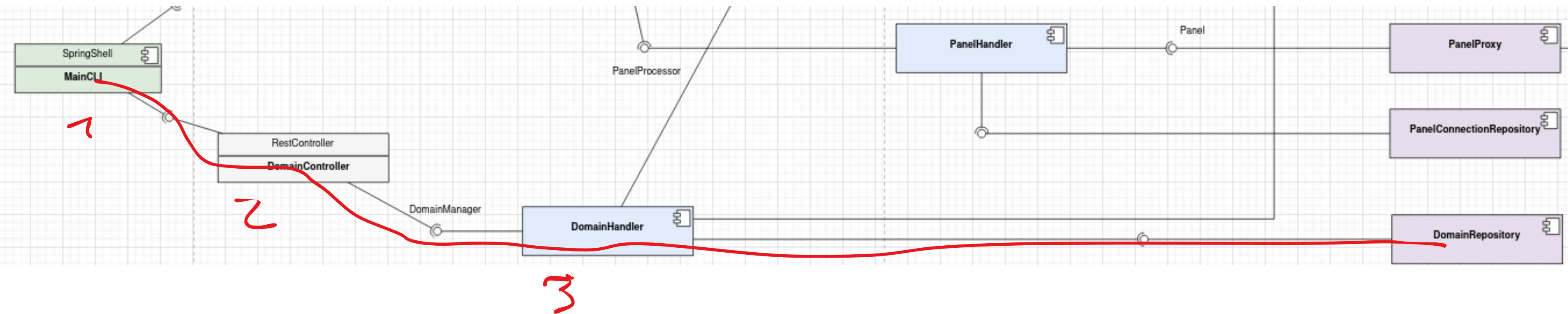
- Mise en place de Thresholds (Warning & Critical)
- On simule via un script externe de nombreux passage sur la gate pour atteindre les thresholds
- Fermeture automatique du portique lorsque le Threshold Critical est atteint
- Re Ouverture de la gate depuis la station de contrôle

Scénario 7:

- Génération et affichage d’un rapport journalier

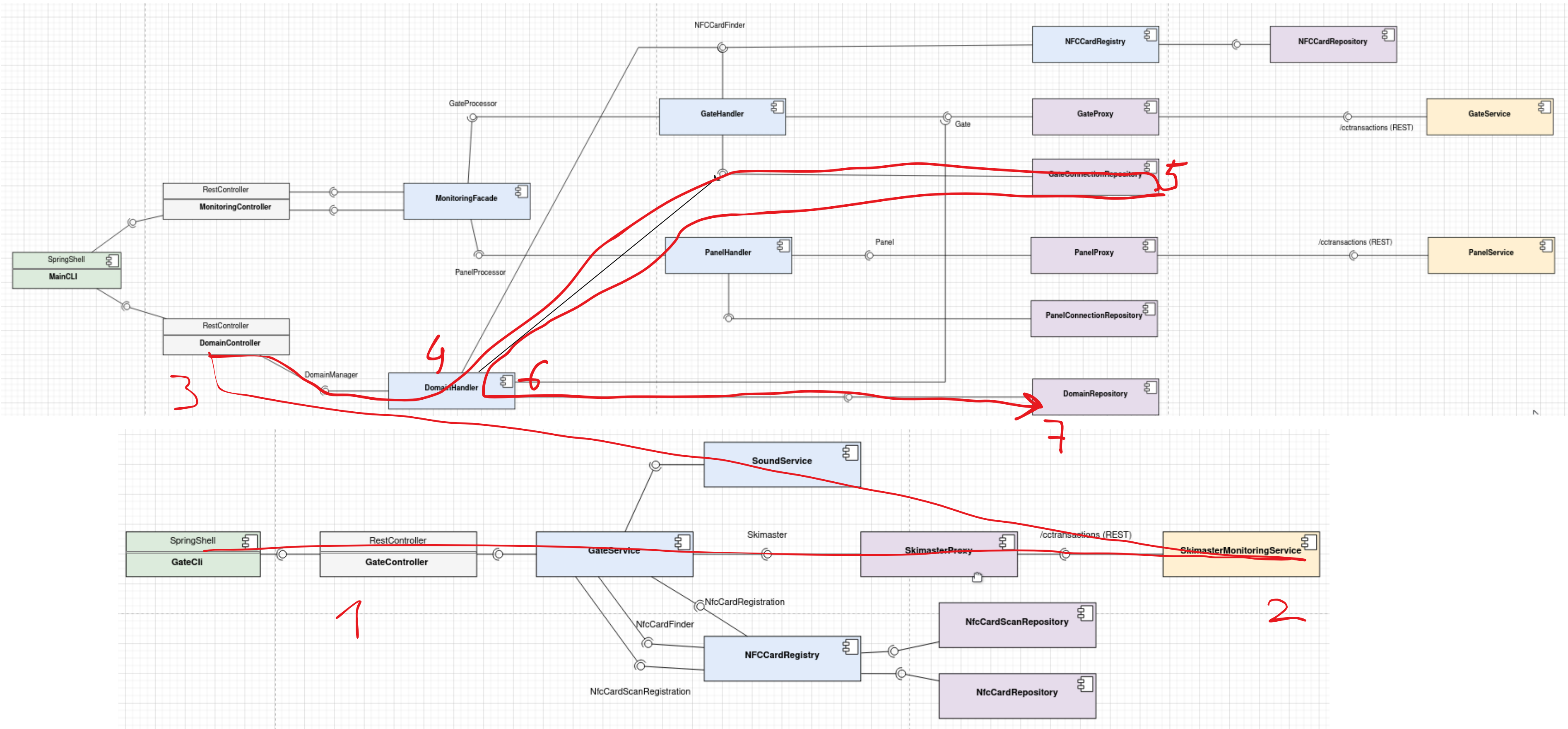
ISA - Scénario 1 : Setup

Setup d'un domaine et ajout d'un plan à un domaine



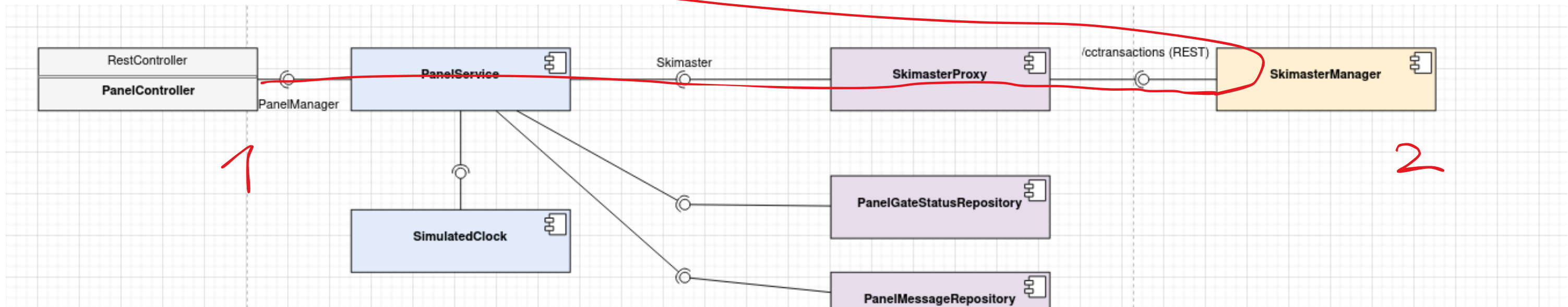
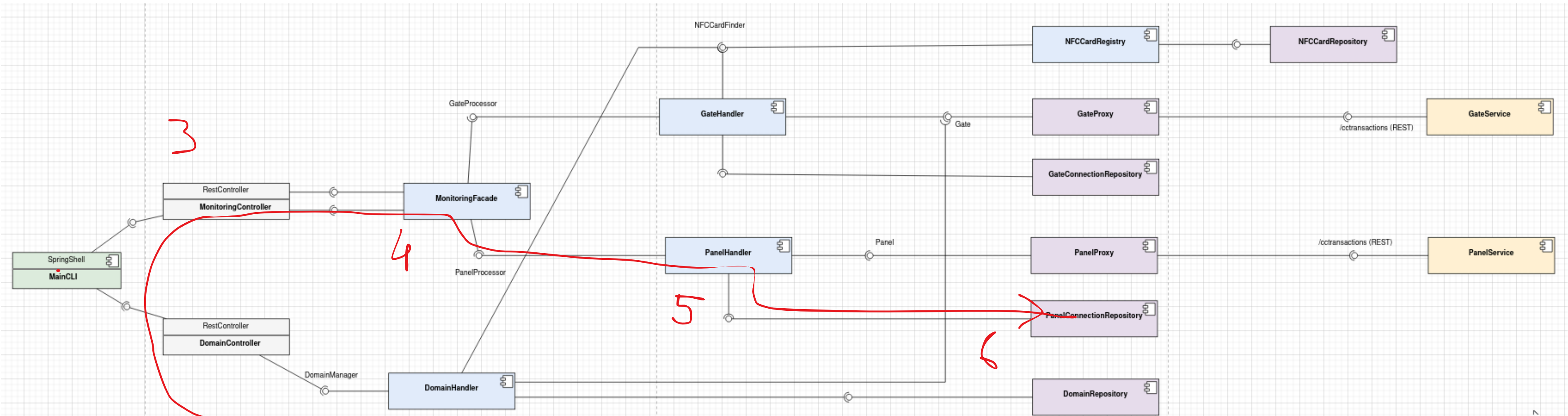
ISA - Scénario 1 : Setup

Enregistrement des portiques



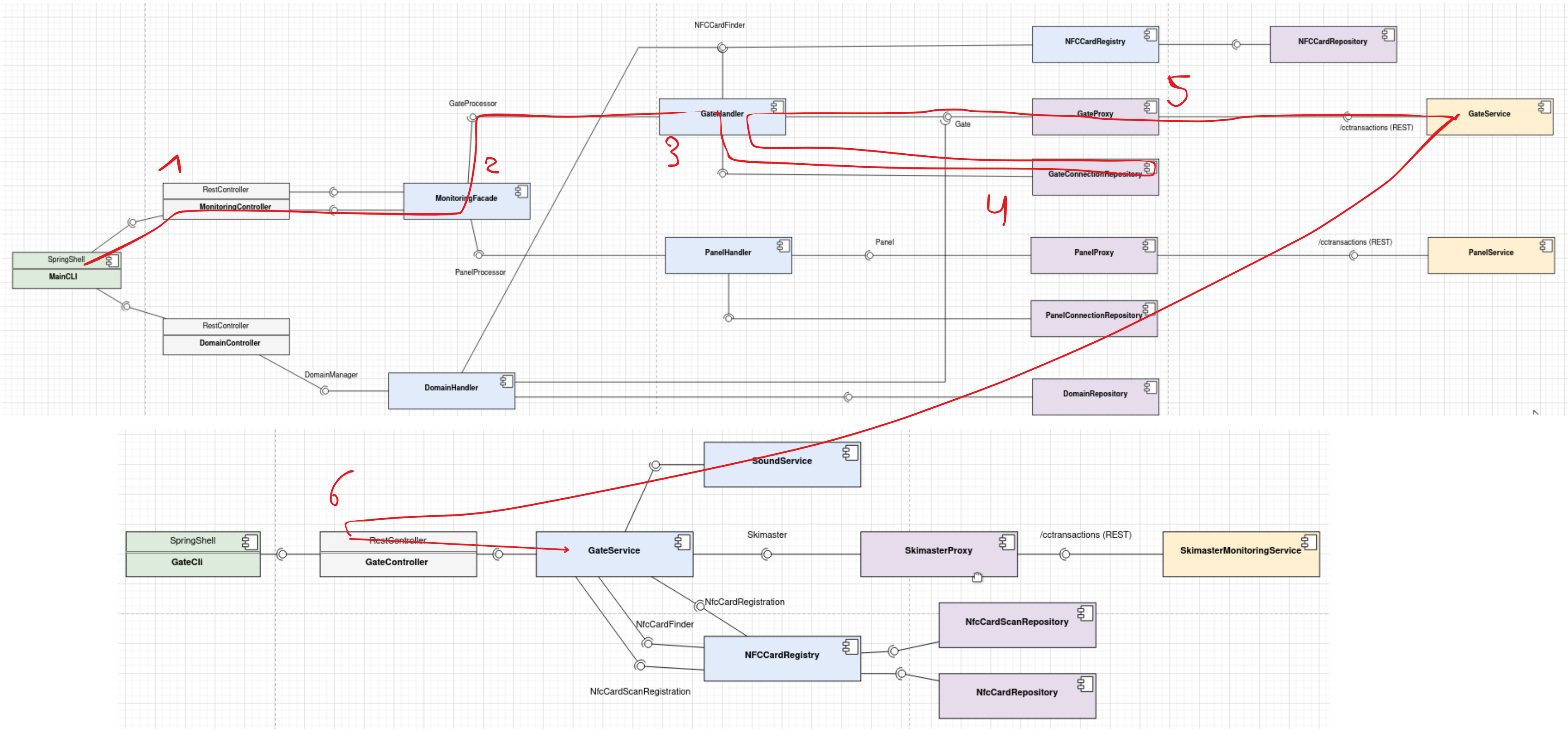
ISA - Scénario 1 : Setup

Enregistrement des panels



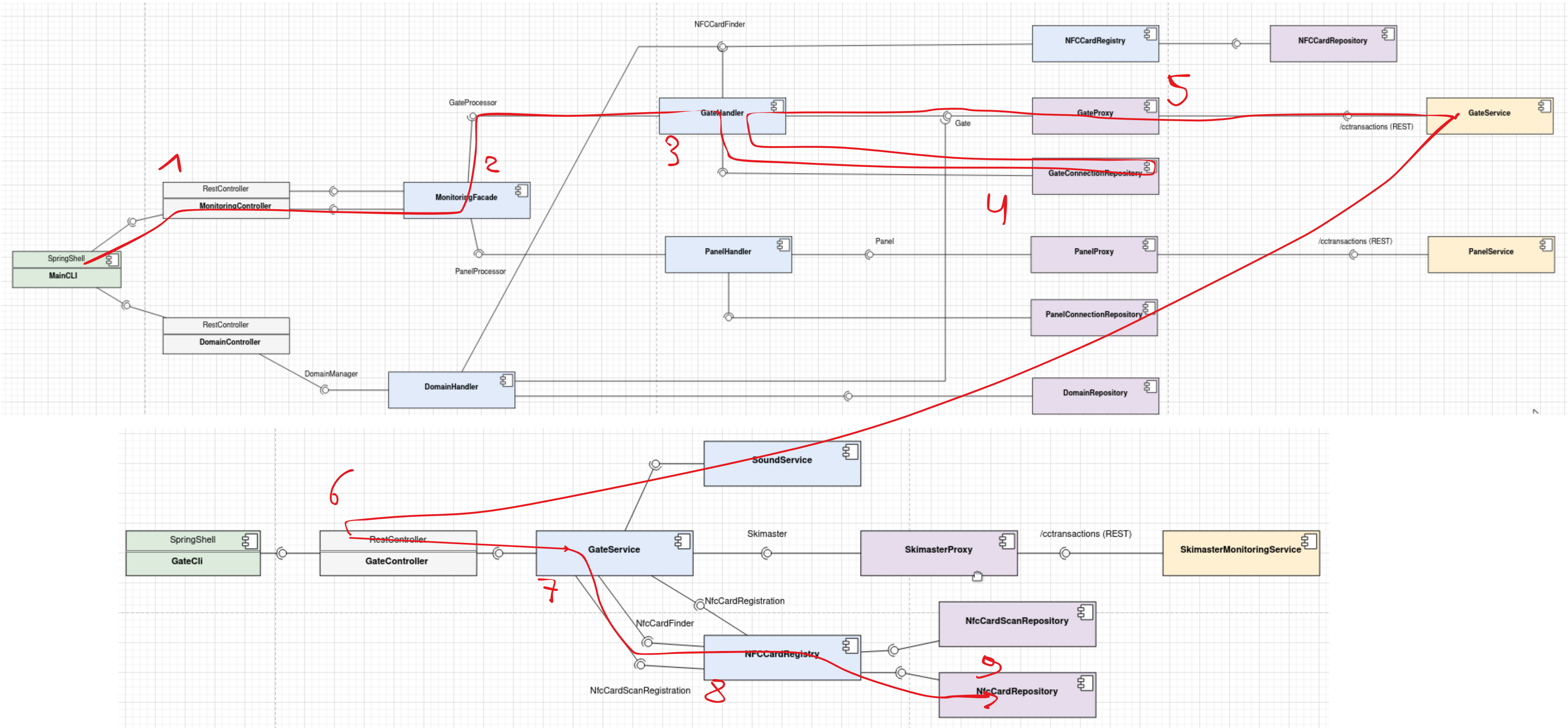
ISA - Scénario 1 : Setup

Overture des portiques

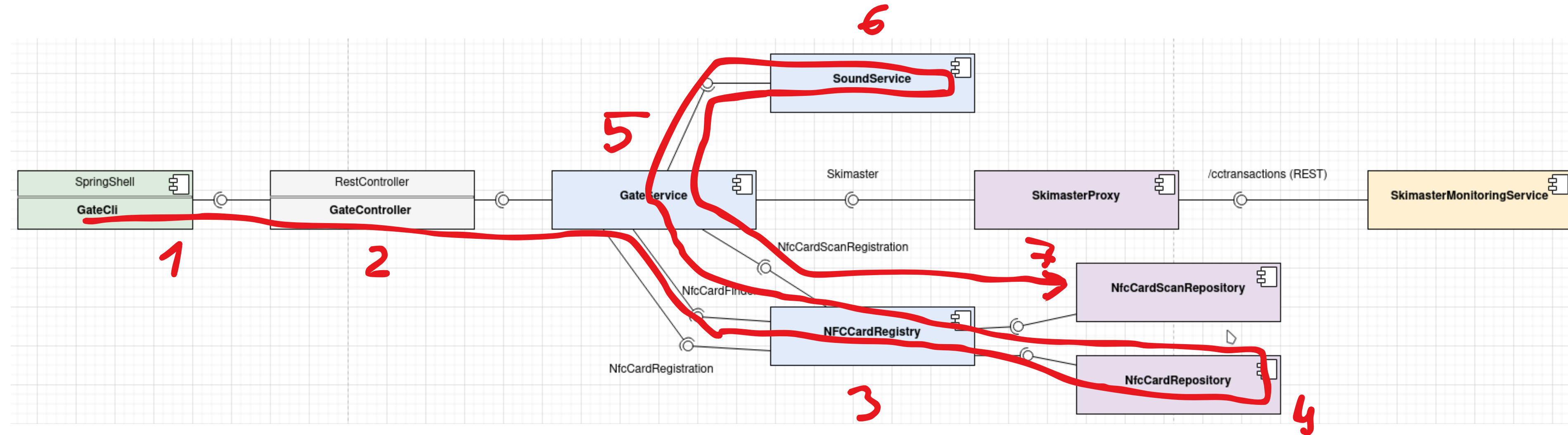


ISA - Scénario 1 : Setup

Envoie des cartes dans les portiques

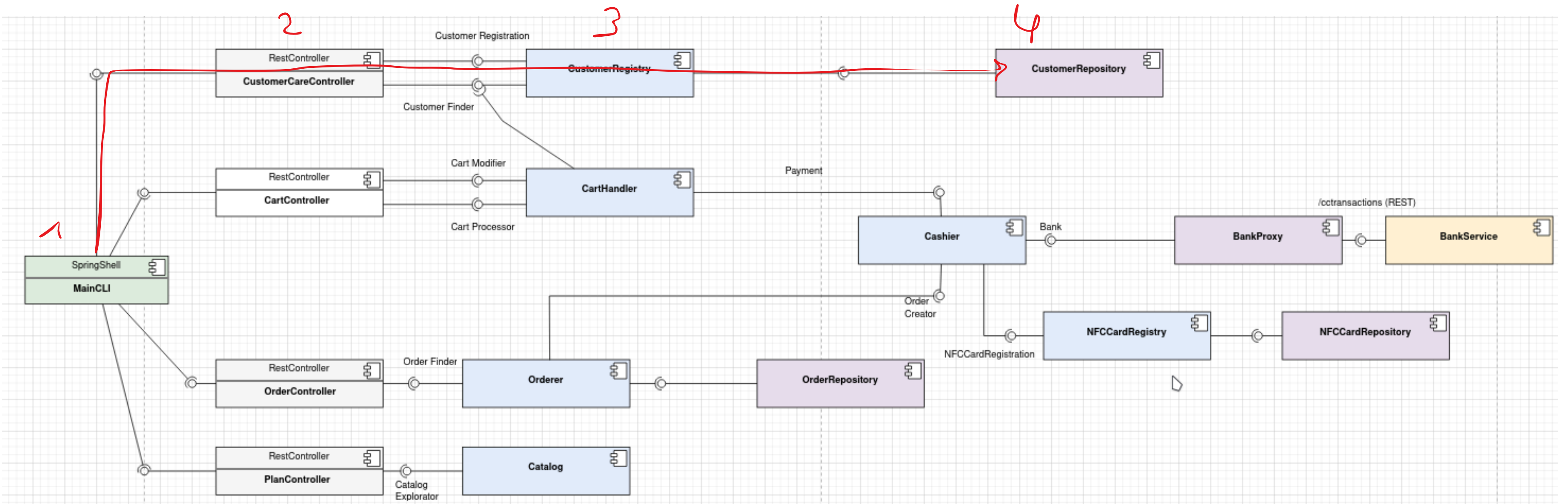


ISA - Scénario 2 : Scan Card



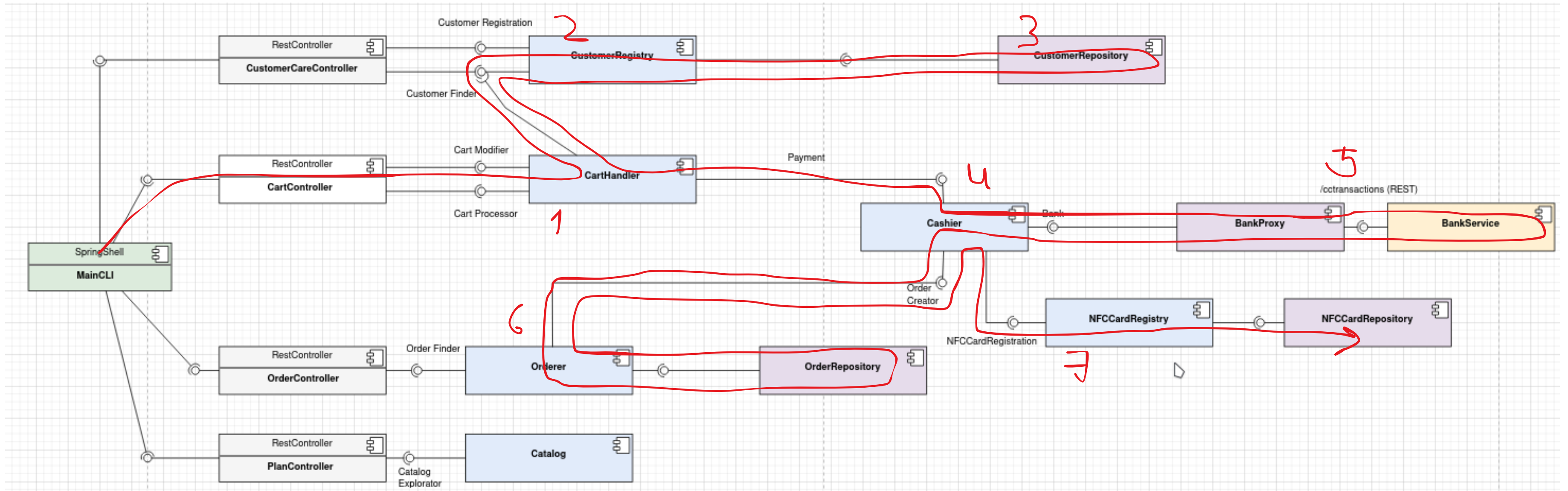
ISA - Scénario 3 : Achat d'un pack famille

Enregistrement d'un customer

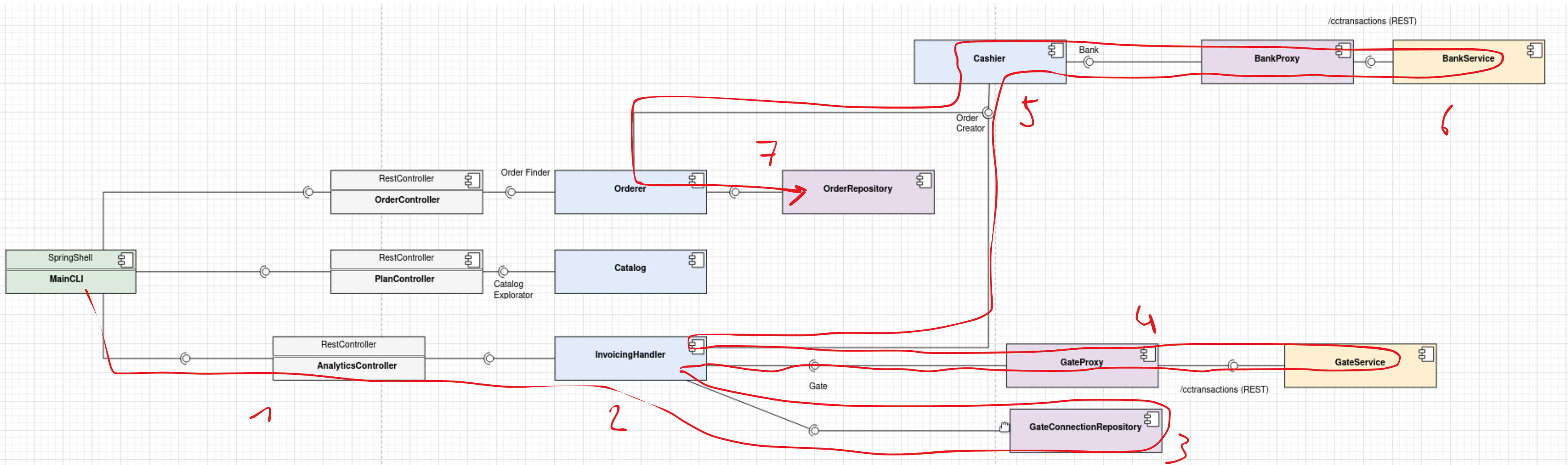


ISA - Scénario 3 : Achat d'un pack famille

Achat d'un pack

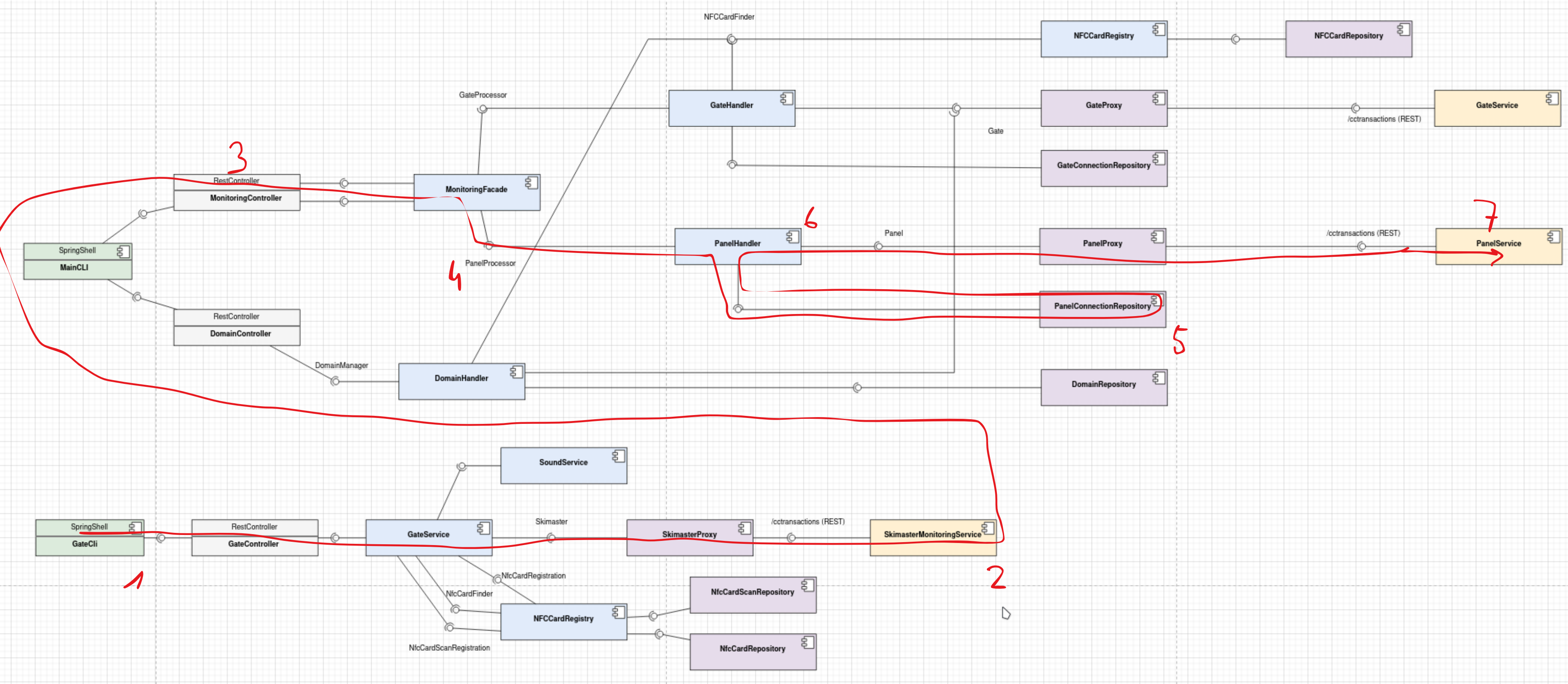


ISA - Scénario 4 : Facturation des super cartes



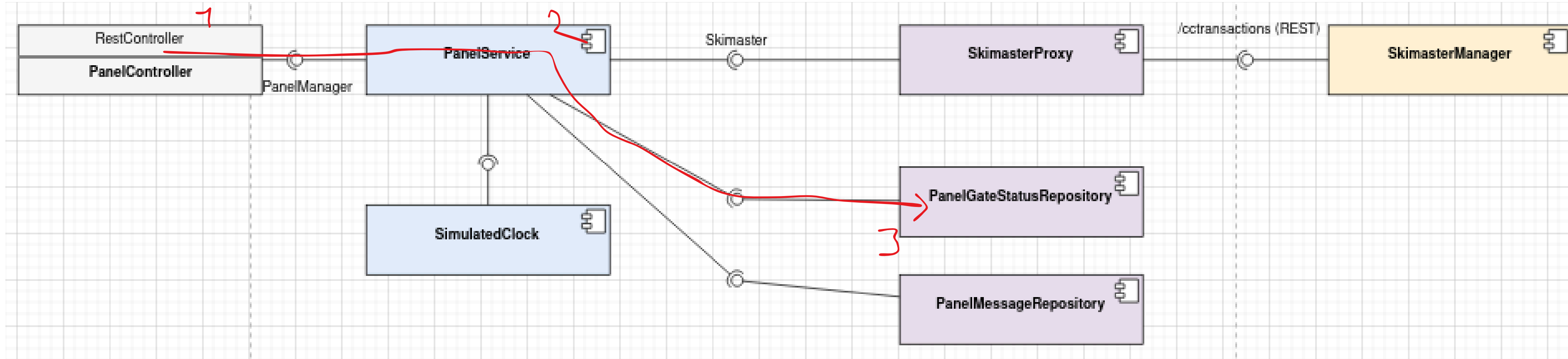
ISA - Scénario 5: Gateman Report

Signalement du perchiste depuis la gate



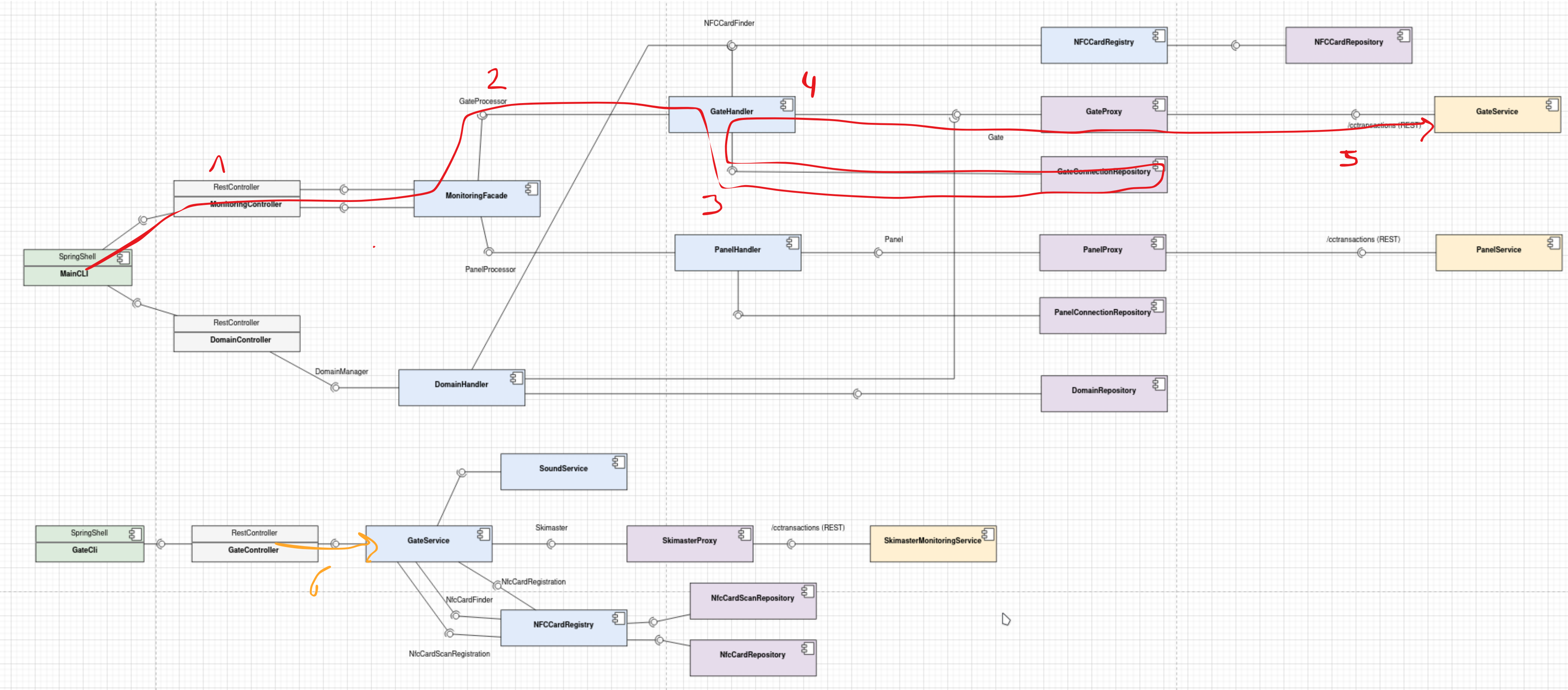
ISA - Scénario 5 : Gateman Report (suite)

Affichage de la fermeture dans le panel



ISA - Scénarios 6 : Gate Threshold

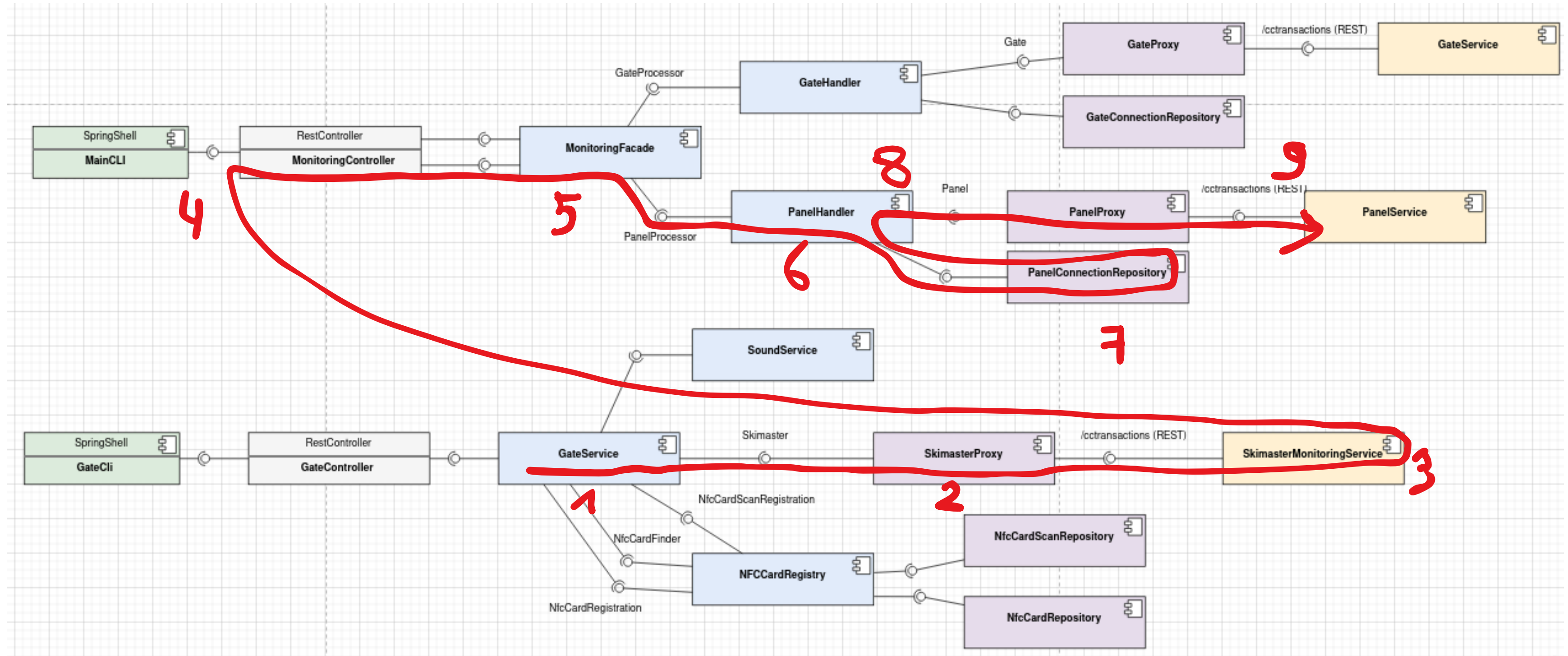
Configuration des thresholds



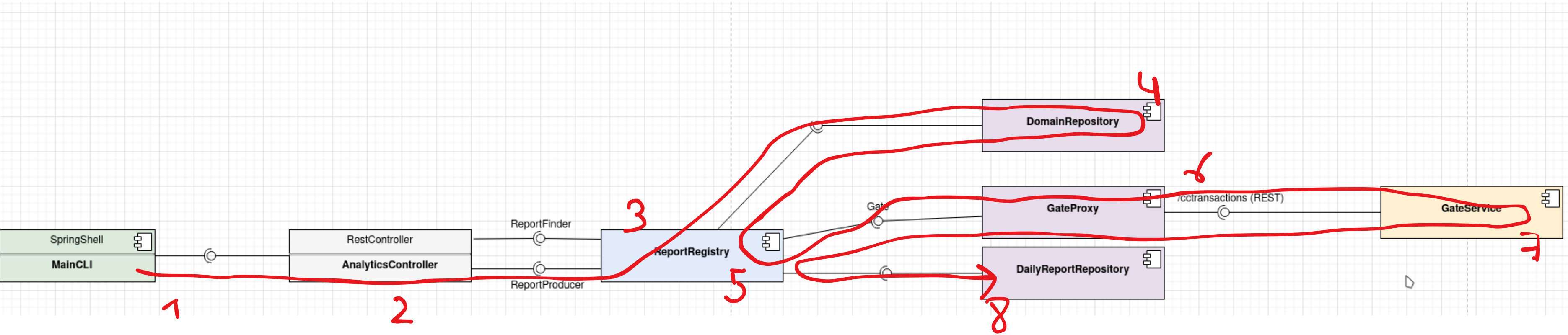
ISA - Scénarios 6 : Gate Threshold

Déclenchement des thresholds et fermeture automatique du portique

On partira du principe que l'on vient de scanner une carte et que le threshold à donc été déclenché (chemin slide 16)



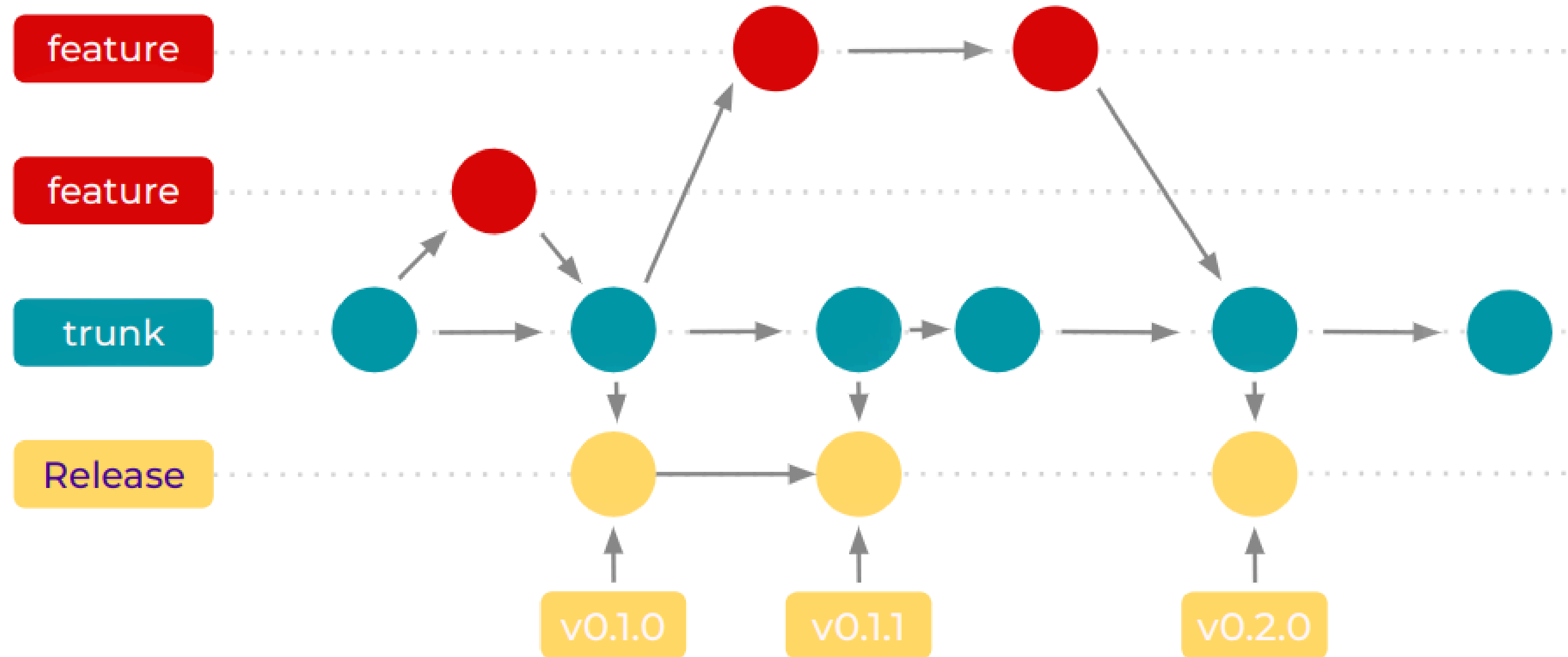
ISA - Scénario : Daily Report



ISA - Sommaire

- **Architecture**
- **Diagramme de composant**
- **Interfaces**
- **Diagramme de classe**
- **Persistance**
- **Scénarios**

DevOps - Stratégie : Trunk Based



DevOps - Pipeline

Afin de démontrer nos pipeline on va prendre l'exemple d'un scénario de dev classique.

Sur notre branche feature/example :

Quand on push, le CI lance :

- Verification des noms des commits
- Lancement des test unitaire

Puis on fait une PR sur dev :

- Lancement des tests d'intégration
- Merge autorisé seulement si 2 personnes au minimum ont approuvé la PR

Si le commit de merge de la PR sur dev contient [major], [minor] ou [patch]

- Déclencher Release-It qui va
 - Créer une PR de dev vers main

DevOps - Pipeline

Si on a une PR sur main qui vient de dev

- On lance les testes e2e.
- Validation obligatoire de 2 personnes pour valider le merge sur main

Si une PR sur main est fermée

- Release-It créé une nouvelle release en fonction du titre de la PR ([major], [minor], [patch])
- On vient build les images docker de backend, bank, gate, cli, gate-cli et panel
- On vient push les images docker sur docker hub

DevOps - Recul

Pipeline CI/CD

Nous avons mis en place une pipeline CI/CD tout au long du projet, ce qui nous a apporté plusieurs bénéfices concrets

- Suivi de la santé du code : chaque push déclenche automatiquement les vérifications, nous permettant d'avoir une vision continue de l'état du projet.
- Remontée rapide des problèmes : en cas de régression ou d'erreur, la pipeline nous alerte immédiatement, limitant ainsi le temps de correction.
- Travail en parallèle : grâce à la séparation des features en branches distinctes, chaque membre de l'équipe pouvait développer indépendamment sans risquer d'impacter le travail des autres.

Évolution de la pipeline

Notre pipeline a évolué au fil du projet. Par exemple, nous avons initialement mis en place une vérification du nommage des commits via le Conventional Commit, uniquement au moment d'une Pull Request sur dev. Nous avons par la suite étendu cette vérification à chaque push sur n'importe quelle branche, afin de pouvoir identifier et corriger les erreurs de nommage au plus tôt.

Concernant les tests e2e, ceux-ci ont été implémentés tardivement dans le projet. En effet, le besoin ne s'est pas fait ressentir dans un premier temps, notre pipeline couvrant suffisamment nos besoins de validation au fil des itérations. Nous sommes néanmoins conscients que ce ressenti d'équipe ne reflète pas nécessairement une bonne pratique, et qu'une intégration plus précoce des tests e2e aurait été préférable.

Repartition des points

- **Driss Meskini : 121**
- **Cyril Rouillon : 121**
- **Dorian Reynier : 120**
- **Léo Migny : 119**
- **Ludovic Clolot : 119**
- **Aurélien Gorjux : 0**